

6. Условный рендеринг

Цель занятия

На прошлых занятиях мы научились создавать компоненты, передавать props, работать с состоянием и выводить списки через `.map()`. Теперь разберем, как показывать разные элементы интерфейса в зависимости от условий.

Условный рендеринг нужен, когда интерфейс должен меняться:

- счетчик равен 0 или уже был нажат;
- подсказка открыта или закрыта;
- данные есть или данных нет;
- элемент активен или неактивен;
- пользователь авторизован или нет.

После занятия должны уметь:

- объяснить, что такое условный рендеринг;
- использовать `if` перед `return`;
- использовать тернарный оператор `? :` внутри JSX;
- использовать `&&` для показа элемента только при выполнении условия;
- понимать, когда компонент может вернуть `null`;
- не ставить число слева от `&&`;
- применять условия к текущему проекту.

1. Что говорит документация React

В актуальной документации React условный рендеринг описывается как обычная JavaScript-логика внутри компонентов. В React нет отдельного специального синтаксиса для условий. Мы используем знакомые инструменты JavaScript:

```
text
  if
  ? :
  &&
  null
```

Коротко:

```
text
  Условие решает, какой JSX показать пользователю.
```

Пример:

```
javascript
function Message({ isVisible }) {
  if (!isVisible) {
    return null
  }

  return <p>Сообщение видно</p>
}
```

Если `isVisible` равно `false`, компонент ничего не отрисует.

2. Что такое условный рендеринг

Условный рендеринг - это показ разных элементов интерфейса в зависимости от условия.

Обычный JavaScript:

```
javascript
if (count === 0) {
  console.log('Кнопку еще не нажимали')
} else {
  console.log('Кнопку уже нажимали')
}
```

React-компонент:

```
javascript
function CounterStatus({ count }) {
  if (count === 0) {
    return <p>Кнопку еще не нажимали</p>
  }

  return <p>Кнопку нажали {count} раз</p>
}
```

Компонент возвращает разный JSX в зависимости от значения count.

3. Условие через `if`

Самый понятный способ - обычный if.

```
javascript
function CounterStatus({ count }) {
  if (count === 0) {
    return <p>Кнопку еще не нажимали</p>
  }

  return <p>Кнопку нажали {count} раз</p>
}
```

Как это читается:

```
text
Если count равен 0, показать первый текст.
Иначе показать второй текст.
```

Такой способ удобен, когда:

- условие большое;
- нужно вернуть сильно разный JSX;

4. Возврат `null`

Иногда компонент должен ничего не показывать.

```
javascript
function Hint({ isVisible }) {
  if (!isVisible) {
    return null
  }

  return <p>Это подсказка</p>
}
```

null означает: компонент ничего не отрисует.

Важно: компонент в React должен что-то вернуть. Если показывать нечего, можно вернуть `null`.

В текущем проекте подсказку можно оформить так:

```
javascript
function Hint({ isVisible }) {
  if (!isVisible) {
    return null
  }

  return (
    <p className="state-hint">
      useState хранит данные, которые влияют на интерфейс.
    </p>
  )
}
```

5. Тернарный оператор `?:`

Тернарный оператор позволяет выбрать одно из двух значений прямо внутри JSX.

```
javascript
condition ? valueIfTrue : valueIfFalse
```

Пример:

```
javascript
<button>
  {isHintVisible ? 'Hide hint' : 'Show hint'}
</button>
```

Как читается:

```
text
Если isHintVisible true, показать "Hide hint".
Иначе показать "Show hint".
```

В нашем проекте это удобно для кнопки подсказки:

```
javascript
<ActionButton
  label={isHintVisible ? 'Hide hint' : 'Show hint'}
  onClick={handleHintClick}
/>
```

Кнопка сама меняет текст в зависимости от состояния.

6. Логический оператор `&&`

Оператор `&&` часто используют, когда нужно показать элемент только при условии.

```
javascript
{isHintVisible && (
  <p>Подсказка открыта</p>
)}
```

Как читается:

```
text
Если isHintVisible true, показать абзац.
Если false, не показывать ничего.
```

Это короче, чем:

```
javascript
{isHintVisible ? <p>Подсказка открыта</p> : null}
```

&& хорошо подходит для ситуаций “показать или не показать”.

7. Важная ошибка с `&&` и числами

Нельзя бездумно ставить число слева от &&.

Неправильно:

```
javascript
{count && <p>Кнопку нажимали</p>}
```

Если count равен 0, React может вывести 0 на страницу.

Правильно:

```
javascript
{count > 0 && <p>Кнопку нажимали</p>}
```

Теперь слева от && находится логическое условие: true или false.

Правило:

```
text
Слева от && лучше писать явное boolean-условие.
```

8. Условие в тексте

Иногда нужно менять не целый блок, а только часть текста.

```
javascript
<p>
  {count === 0
    ? 'Кнопку еще не нажимали'
    : `Кнопку нажали ${count} раз`}
</p>
```

Для короткого текста тернарный оператор подходит хорошо.

Но если JSX становится длинным и трудно читаемым, лучше вынести логику в отдельный компонент.

9. Условие в className

Иногда условие влияет на CSS-класс.

```
javascript
<p className={count > 0 ? 'state-hint active' : 'state-hint'}>
  Состояние счетчика
</p>
```

Если count > 0, будет класс:

```
text
state-hint active
```

Если count равен 0, будет:

```
text
state-hint
```

В больших проектах для классов часто используют специальные библиотеки, но на первых занятиях достаточно тернарного оператора.

10. Условный компонент для статуса счетчика

Создадим компонент:

```
javascript
function CounterStatus({ count }) {
  if (count === 0) {
    return <p className="counter-status">Кнопку еще не нажимали.</p>
  }

  if (count < 5) {
    return (
      <p className="counter-status">
        Кнопку нажали {count} раз. Продолжайте.
      </p>
    )
  }

  return (
    <p className="counter-status success">
      Отлично, счетчик дошел до {count}.
    </p>
  )
}
```

Здесь три варианта:

```
text
count === 0  -> первый текст
count < 5    -> второй текст
иначе       -> третий текст
```

Этот пример хорошо показывает, когда `if` читается лучше, чем длинный тернарный оператор.

11. Условная подсказка отдельным компонентом

Сейчас подсказку можно показывать через `&&`:

```
javascript
{isHintVisible && (
  <p className="state-hint">
    useState хранит данные, которые влияют на интерфейс.
  </p>
)}
```

А можно вынести в компонент:

```
javascript
function StateHint({ isVisible }) {
  if (!isVisible) {
    return null
  }

  return (
    <p className="state-hint">
      useState хранит данные, которые влияют на интерфейс.
    </p>
  )
}
```

Использование:

```
javascript
  <StateHint isVisible={isHintVisible} />
```

Так код в App становится короче, а тема `return null` становится видна в проекте.

12. Сравнение способов

Способ	Когда использовать
<code>if</code>	Когда нужно вернуть сильно разные варианты JSX или условие сложное.
<code>return null</code>	Когда компонент иногда не должен ничего показывать.
<code>? :</code>	Когда нужно выбрать один из двух вариантов прямо в JSX.
<code>&&</code>	Когда нужно показать элемент только при <code>true</code> .

Главное: выбирайте тот способ, который легче читать.

13. Практический пример для проекта

Добавим статус счетчика:

```
javascript
function CounterStatus({ count }) {
  if (count === 0) {
    return <p className="counter-status">Кнопку еще не нажимали.</p>
  }

  if (count < 5) {
    return (
      <p className="counter-status">
        Кнопку нажали {count} раз. Продолжайте.
      </p>
    )
  }

  return (
    <p className="counter-status success">
      Отлично, счетчик дошел до {count}.
    </p>
  )
}
```

Используем его в App:

```
javascript
  <CounterStatus count={count} />
```

Теперь интерфейс объясняет состояние счетчика.

14. Практический пример: текст кнопки подсказки

Сейчас можно сделать кнопку более понятной:

```
javascript
<ActionButton
  label={isHintVisible ? 'Hide hint' : 'Show hint'}
  onClick={handleHintClick}
/>
```

Если подсказка открыта, кнопка предлагает скрыть ее. Если закрыта - показать.

Это пример тернарного оператора внутри props.

15. Частые ошибки

Ошибка 1. Использовать `if` прямо внутри JSX

Неправильно:

```
javascript
return (
  <div>
    {if (count > 0) <p>Есть клики</p>}
  </div>
)
```

Правильно вынести if до return:

```
javascript
if (count > 0) {
  return <p>Есть клики</p>
}
```

Или использовать &&:

```
javascript
{count > 0 && <p>Есть клики</p>}
```

Ошибка 2. Ставить число слева от `&&`

Неправильно:

```
javascript
{count && <p>Есть клики</p>}
```

Правильно:

```
javascript
{count > 0 && <p>Есть клики</p>}
```

Ошибка 3. Слишком длинный тернарный оператор

Плохо читается:

```
javascript
{count === 0 ? <p>Нет кликов</p> : count < 5 ? <p>Мало кликов</p> : <p>Много кликов</p>}
```

Лучше вынести в компонент с if:

```
javascript
  <CounterStatus count={count} />
```

Ошибка 4. Забыть вернуть JSX

Неправильно:

```
javascript
function Hint({ isVisible }) {
  if (!isVisible) {
    return null
  }
}
```

Если `isVisible` равно `true`, компонент ничего не возвращает.

Правильно:

```
javascript
function Hint({ isVisible }) {
  if (!isVisible) {
    return null
  }

  return <p>Подсказка</p>
}
```

16. Практика

Задание 1

Создайте компонент `CounterStatus`:

```
javascript
function CounterStatus({ count }) {
  if (count === 0) {
    return <p className="counter-status">Кнопку еще не нажимали.</p>
  }

  if (count < 5) {
    return (
      <p className="counter-status">
        Кнопку нажали {count} раз. Продолжайте.
      </p>
    )
  }

  return (
    <p className="counter-status success">
      Отлично, счетчик дошел до {count}.
    </p>
  )
}
```


Задание 2

Добавьте компонент в App:

```
javascript
  <CounterStatus count={count} />
```

Задание 3

Измените кнопку подсказки:

```
javascript
  <ActionButton
    label={isHintVisible ? 'Hide hint' : 'Show hint'}
    onClick={handleHintClick}
  />
```

Задание 4

Создайте компонент StateHint:

```
javascript
function StateHint({ isVisible }) {
  if (!isVisible) {
    return null
  }

  return (
    <p className="state-hint">
      useState хранит данные, которые влияют на интерфейс.
    </p>
  )
}
```

Задание 5

Проверьте:

- при `count = 0` показывается один текст;
- при `count` от 1 до 4 показывается другой текст;
- при `count >= 5` показывается третий текст;
- кнопка подсказки меняет текст;
- подсказка появляется и исчезает.

17. Контрольные вопросы

1. Что такое условный рендеринг?
2. Какие JavaScript-инструменты используются для условий в React?
3. Когда удобно использовать `if`?
4. Что означает `return null`?
5. Когда удобно использовать тернарный оператор `? :?`
6. Когда удобно использовать `&&`?
7. Почему нельзя писать `{count && <p>...</p>}`?
8. Как правильно написать условие с числом слева от `&&`?
9. Почему слишком длинные тернарные операторы плохо читаются?
10. Как условие может изменить текст кнопки?
11. Как условие может изменить CSS-класс?
12. Почему иногда лучше вынести условие в отдельный компонент?
13. Что вернет компонент, если не написать `return` для одного из вариантов?

14. Чем `?` : отличается от `&&`?

15. Где в нашем проекте уже использовался условный рендеринг?

18. Краткий итог

Условный рендеринг позволяет показывать разные элементы интерфейса в зависимости от состояния, props или других данных.

Главные способы:

text

<code>if</code>	-> удобно для больших условий
<code>null</code>	-> ничего не показывать
<code>? :</code>	-> выбрать один из двух вариантов
<code>&&</code>	-> показать элемент только если условие true

Главная мысль:

text

В React условия пишутся обычным JavaScript.